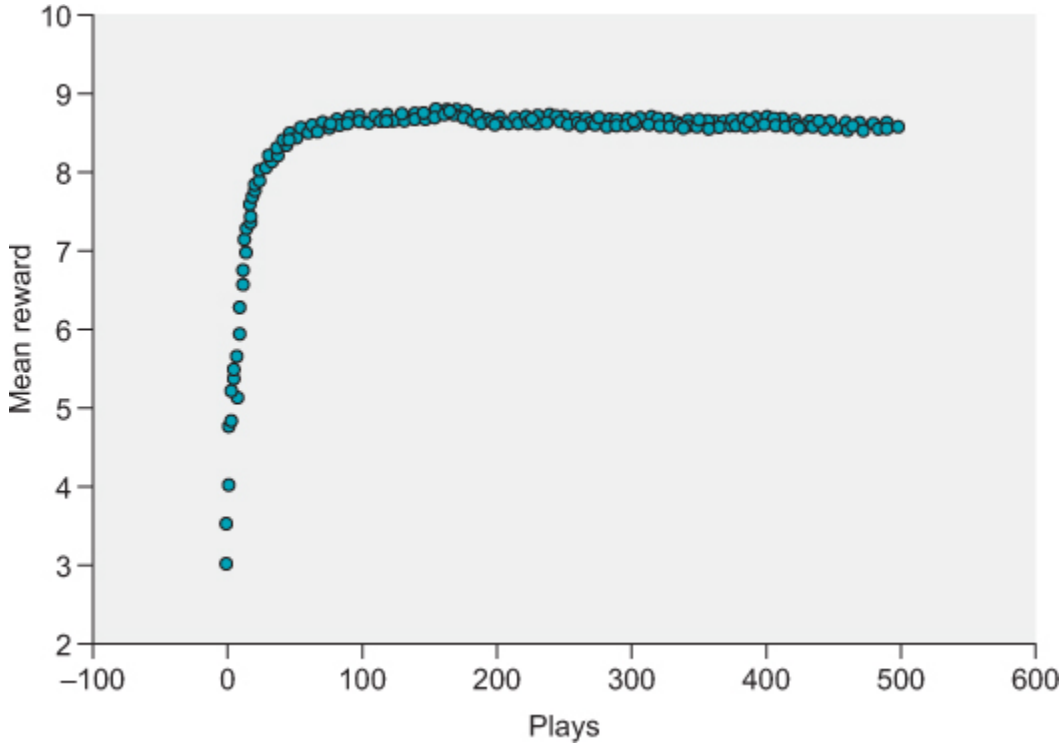
```
    rewards.append(mean_reward)
ax.scatter(np.arange(len(rewards)),rewards)
```

- ***1*** Computes softmax probabilities for each arm with respect to their current action values
- ***2*** Chooses an arm randomly but weighted by the softmax probabilities

Softmax action selection seems to do better than the epsilon-greedy method for this problem as you can tell from figure 2.4; it looks like it converges on an optimal policy faster. The downside to softmax is having to manually select the τ parameter. Softmax here was pretty sensitive to τ, and it takes some time playing with it to find a good value. Obviously with epsilon-greedy we had to set the epsilon parameter, but choosing that parameter was much more intuitive.

Figure 2.4. With the softmax policy, the n-armed bandit algorithm tends to converge faster on the maximal mean reward.



2.3. Applying bandits to optimize ad placements